

CSE 3212
Compiler Design Laboratory

Project Report

Sprach

German-inspired Programming Language

Assigned to:

Asique Ehetasamul Haque

Roll: **2107096**

Year: 3rd

Semester: 2nd

Group: B2

Department of Computer Science and Engineering,
Khulna University of Engineering and Technology

Description

This project implements a **German-inspired mini programming language** developed using **Flex and Bison**. The compiler supports essential programming constructs, including variable declarations, functions, control structures, and mathematical operations, utilizing German keywords and syntax.

Language features

- ❖ **Data Types:**
Supports integers, doubles, strings, characters, and arrays.
- ❖ **Array Operations:**
Includes Object-Oriented style operations such as appending, popping, and sorting.
- ❖ **Functions:**
User-defined functions with parameters and return types.
- ❖ **Control Flow:**
Supports wenn-sonst (if-else), fuer (for) loops, waehrend (while) loops, and wahl (switch) cases.
- ❖ **Comments:**
Single-line (#) and multi-line (## ... ##) comment features.
- ❖ **Operations:**
Full support for arithmetic, logical, relational, and bitwise operations.
- ❖ **I/O:**
Interactive input and formatted output

Variable Naming Conventions

Rule	Description	Examples
Starting Character	Must start with an underscore or a letter	meinname, _temp
Subsequent Characters	Can contain letters, digits, and underscores	mein_name, roll123
Case Sensitivity	The language is case-sensitive	Name != name

Keywords and Control Structures

German Keyword	C Equivalent
wenn	if
sonst wenn	else if
sonst	else
für	for
in	in
während	while
wahl	switch
fall	case
standard	default
leere	void

Data Types

German Keyword	C Equivalent
GANZ	int
KOMM	double
TEXT	string
ZEICHEN	char

Operators

Logical Operators

- ❖ und (&&)
- ❖ oder (||)
- ❖ nicht (!)

Mathematical functions

German Function	C Equivalent
mathe_sin	<i>sin()</i>
mathe_cos	<i>cos()</i>
mathe_tan	<i>tan()</i>
quadrat_wurzel	<i>sqrt()</i>
mathe_log	<i>log()</i>

Input-Output & Array Operations

Operation	German Keyword	C/C++ Equivalent
Input	lesen	scanf()
Output	geben	printf()
Add to Array	hinzufuegen	push_back()
Remove Last	entfernen	pop_back()
Sort	sortieren	sort()

Intermediate Code Generation with TAC (Three Address Code)

What is TAC?

TAC (Three Address Code) is an intermediate representation of source code that bridges the gap between high-level programming and machine instructions. It breaks down complex expressions into simple, sequential operations where each instruction has at most three operands (operand 1, operand 2, result).

Why is TAC used?

- **Clarity:** Shows exactly what the compiler does with each operation within a file.
- **Optimization Ready:** Makes it easier to apply compiler optimizations
- **Machine Generation:** Can be easily converted to assembly or machine code
- **Debugging:** Helps understand program flow and transformations

TAC Example:

For the SPRACH program:

```
GANZ a = 5;  
GANZ b = 3;  
GANZ c = a + b;  
geben c;
```

The compiler generates this TAC inside ***tac_output.txt***:

===== INTERMEDIATE CODE (THREE-ADDRESS CODE) =====

```
[ 0] _t0 = CONST 5  
[ 1] STORE a = _t0  
[ 2] _t1 = CONST 3  
[ 3] STORE b = _t1  
[ 4] _t2 = LOAD a  
[ 5] _t3 = LOAD b  
[ 6] _t4 = _t2 + _t3  
[ 7] PRINT _t4
```

What This Shows:

- Line [0-1]: Load and store constant 5 to variable 'a'
- Line [2-3]: Load and store constant 3 to variable 'b'
- Line [4-5]: Load variables a and b into temporary registers
- Line [6]: Perform addition, store result in _t4
- Line [7]: Print the result (8)

Noticeably here:

- Temporary variables (`\_t0`, `\_t1`, etc.) store intermediate results
- Each operation is atomic (one instruction)
- Complex expressions are decomposed into steps
- Variable loads and stores are explicit